

El framework spring mvc

M6: Desarrollo de aplicaciones jee con spring framework

|AE2: Utilizar framework spring boot y spring mvc para la implementación de una aplicación web básica de acuerdo a las buenas prácticas de la industria

Introducción

Spring Framework es una plataforma que actúa de soporte para desarrollar aplicaciones Java. Maneja toda la infraestructura y así te puedes centrar en tu aplicación. Diciéndolo más coloquialmente, Spring es el “**pegamento**” que une todos los componentes de la aplicación, maneja su ciclo de vida y la interacción entre ellos.

Spring Boot es un poderoso framework de desarrollo de aplicaciones basado en Spring Framework. Diseñado para simplificar la configuración y el desarrollo de aplicaciones Java, Spring Boot proporciona un enfoque de convención sobre configuración que minimiza la cantidad de código y configuración necesarios.

Una de las características clave de Spring Boot es su capacidad para empaquetar aplicaciones en un archivo JAR ejecutable o un archivo WAR tradicional, lo que facilita el despliegue y la distribución de las aplicaciones en diferentes entornos.

También aprenderás a implementar la arquitectura MVC a través de **Spring MVC**, que permite una separación clara de las responsabilidades. El modelo se encarga de representar los datos y la lógica del negocio, la vista se encarga de la presentación de los datos al usuario y el controlador actúa como el intermediario que maneja las solicitudes del usuario, invoca la lógica del negocio apropiada y selecciona la vista adecuada para mostrar los resultados.

Aprendizaje esperado

Cuando finalices la lección podrás:

- Utilizar framework Spring Boot y Spring MVC para la implementación de una aplicación web básica de acuerdo a las buenas prácticas de la industria

Spring Boot

Si bien es cierto que Spring Framework es muy potente, la configuración inicial y la preparación de las aplicaciones para producción son tareas bastante tediosas. **Spring Boot simplifica el proceso** al máximo gracias a sus dos principales mecanismos.

Contenedor de aplicaciones integrado

Spring Boot permite compilar nuestras aplicaciones Web como un archivo **.jar** que podemos ejecutar como una aplicación Java normal (como alternativa a un archivo **.war**, que se despliega en un servidor de aplicaciones como Tomcat).

Esto lo consigue integrando el servidor de aplicaciones en el propio **.jar** y levantándolo cuando arrancamos la aplicación. De esta forma, podemos distribuir nuestras aplicaciones de una forma mucho más sencilla, al poder configurar el servidor junto con la aplicación. Esto también es muy útil en arquitecturas de **microservicios**, puesto que permite distribuir nuestras aplicaciones como imágenes Docker que podemos escalar horizontalmente (algo muy complicado con un **.war**).

Starters

Spring Boot nos proporciona una serie de dependencias, llamadas starters, que podemos añadir a nuestro proyecto dependiendo de lo que necesitemos: crear un controlador REST, acceder a una base de datos usando JDBC, conectar con una cola de mensajes Apache ActiveMQ, etc.

Una vez añadimos un **starter**, éste nos proporciona todas las dependencias que necesitamos, tanto de Spring como de terceros. Además, los starters vienen configurados con valores por defecto, que pretenden minimizar la necesidad de configuración a la hora de desarrollar. Un buen ejemplo es el de **spring-boot-starter-actuator**: una vez que añadimos la dependencia, nuestra aplicación empezará a generar métricas tanto de la JVM como de la aplicación en sí (latencias, errores, etc).

Al igual que con Spring Framework, cualquier configuración puede ser modificada de ser necesario: desde el puerto en el que la aplicación escucha peticiones, hasta el banner que sale por consola al arrancar la aplicación.

Spring Boot centra su éxito en las siguientes características que lo hacen extremadamente fácil de utilizar:

- **Configuración:** cuenta con un complejo módulo que configura todos los aspectos de nuestra aplicación para poder simplemente ejecutarla, sin tener que definir absolutamente nada.
- **Resolución de dependencias:** solo hay que determinar qué tipo de proyecto estaremos utilizando y Spring Boot se encarga de resolver todas las librerías/dependencias para que la aplicación funcione.
- **Despliegue:** se puede ejecutar como una aplicación Stand-alone, pero también es posible ejecutar aplicaciones web, ya que es permite desplegar las aplicaciones mediante un servidor web integrado, como es el caso de Tomcat, Jetty o Undertow.
- **Métricas:** Por defecto, Spring Boot cuenta con servicios que permite consultar el estado de salud de la aplicación, permitiendo saber si la aplicación está prendida o apagada, memoria utilizada y disponible, número y detalle de los Bean's creado por la aplicación, controles para el prendido y apagado, etc.
- **Extensible:** Spring Boot permite la creación de complementos, los cuales ayudan a que la comunidad de Software Libre cree nuevos módulos que faciliten aún más el desarrollo.

Anotaciones

Una anotación es un **metadato que se incrusta en el código**. Se caracterizan por iniciar con el carácter **@**, este no cambia el comportamiento del código pero sí puede indicarle al compilador acciones específicas de lectura o ejecución, por ejemplo, Java por defecto tiene varias anotaciones, una de las más conocidas es **@Override** que se utiliza para asegurar que un método heredado se sobrescriba. Así como ésta hay muchas más y además se pueden crear anotaciones propias.

Spring Framework tiene varias anotaciones por defecto, que nos facilitaran el desarrollo del código desde el punto de vista de reciclaje y composición. La utilización es igual a las anotaciones de Java, se coloca el carácter **@** y el nombre de la anotación arriba de la declaración del método.

Spring se vale de anotaciones para indicarle a la interfaz **BeanPostProcessor** como instanciar los beans.

Inyección de dependencias

La inyección de dependencias es quizás la característica más destacable del core de Spring Framework. Es un **patrón de software para resolver las dependencias de objetos con otros objetos**. Es una forma de implementar la Inversión de Control. Esto significa que en vez de tener que crear los objetos de forma manual, el framework será responsable de crearlos e inyectarlos en las clases u objetos que lo necesiten.

La DI (Dependency Injector o Inyector de Dependencias) consiste en que en lugar de que sean las clases las encargadas de crear (instanciar) los objetos que van a usar (sus atributos), los objetos se inyectaran mediante los métodos setters o mediante el constructor en el momento en el que se cree la clase y cuando se quiera usar la clase en cuestión ya estará lista.

La implementación de la DI se realiza mediante la anotación **@Autowired**, que le indica a Spring dónde debe ocurrir una inyección.

Contenedor Spring

En Spring hay un Contenedor DI que es el encargado de **inyectar a cada objeto los objetos que necesita** (de los que depende) **según se le indique ya sea en un archivo de configuración XML o mediante anotaciones**. En el caso de Spring ese objeto es el contenedor IoC el cual está provisto por los módulos spring-core y spring-beans.

Spring se basa en el principio de Inversión de Control (IoC) o Patrón Hollywood («No nos llames, nosotros le llamaremos») consiste en:

- Un Contenedor que maneja los objetos por vos, este contenedor es un archivo XML. Este archivo se llama application-context.xml.
- El contenedor generalmente controla la creación de estos objetos. Por decirlo de alguna manera, el contenedor se encarga de crear los “new” de las clases java para que no los realices de manera manual.
- El contenedor resuelve dependencias entre los objetos que contiene.

¿Qué es un Bean?

Un Bean es una clase de Java que debe cumplir los siguientes requisitos:

- Tener todos sus atributos privados (private).
- Tener métodos set() y get() públicos de los atributos privados que nos interese.
- Tener un constructor público por defecto.

A diferencia de los Bean convencionales que representan una clase, la particularidad de **los Beans de Spring es que son objetos creados y manejados por el contenedor Spring**.

A continuación veremos la declaración y utilización de algunas de las **principales anotaciones de spring**.

@Configuration: se utiliza para la configuración basada en anotaciones de Spring. @Configuration es una anotación de marcador que indica que una clase declara uno o más beans, y puede ser procesada por el contenedor Spring para generar definiciones de beans y solicitudes de servicio para esos beans en tiempo de ejecución. Esta anotación se importa desde `org.springframework.context.annotation.Configuration`.

Por lo general, la clase que define el método Main es un buen candidato como principal @Configuration. De todas formas, recomendamos utilizar la anotación @Configuration en una clase exclusiva de configuraciones, donde también tendremos las configuraciones de seguridad de nuestro proyecto.

@Configuration

```
public class AppConfig{
}
```

@Bean: se utiliza a nivel de método. La anotación @Bean funciona con @Configuration para crear beans Spring. Como se mencionó anteriormente, @Configuration tendrá métodos para crear instancias y configurar dependencias. Dichos métodos serán anotados con @Bean. El método anotado con esta anotación funciona como la ID del bean, y crea y devuelve el bean real.

@Configuration

```
public class AppConfig {
    @Bean
    public Person person() {
        return new Person(address());
    }
}
```

```
@Bean
public Address address() {
    return new Address();
}
}
```

@Autowired le indica a Spring dónde debe ocurrir una inyección. Si se lo coloca en un método, por ejemplo: set Address, entiende (por el prefijo que establece la anotación **@Autowired**) que se necesita inyectar un bean. Spring busca un bean de tipo Address y, si lo encuentra, lo inyecta en este método. Sustituye la declaración de los atributos del bean en el xml.

@Autowired se emplea para generar la inyección de dependencia de un tipo de Objeto que pertenece a una clase con la Component(**@Controller**, **@Service**, **@Repository**)

```
@Autowired
private final AddressService addressService;
```

@Component le indica a Spring Boot que debe tratar la clase con el decorador como un bean, es decir que el propio motor se encargará de hacer la inyección.

```
@Component
public class MyClass {
}
```

@Repository, **@Service** y **@Controller** son especializaciones de **@Component**.

@Repository se utiliza para la capa de persistencia, en otras palabras, clases que interactúan con la base de datos. Tiene la particularidad de que tiene un control de errores más específico para interactuar con la sintaxis de base de datos, causando un mejor manejo de estos objetos.

@Service se utiliza en la capa de servicio, es decir en la clase que se espera que sea la que maneje la lógica de negocio.

@Controller nos enlaza con la capa de presentación (vistas), allí solemos colocar las urls que se exponen vía web.

Creando un proyecto web de Spring Boot

Al igual que en cualquier biblioteca Java estándar, en Spring Boot se incluyen los correspondientes archivos JAR (Java Archive) o WAR (Web Application Archive) en el Classpath. Java recurre a esta ruta del sistema para buscar los archivos ejecutables. Puedes crear los archivos de almacenamiento de Spring Boot de dos maneras distintas:

- Instala y utiliza Maven para crear por tu cuenta todo el marco del proyecto, incluidas las dependencias necesarias.
- Utiliza el servicio web **Spring Initializr** para establecer la configuración de Spring Boot y, después, descárgala como plantilla de proyecto final.

Una característica de la segunda solución es que proporciona una interfaz web muy fácil de usar para crear los archivos JAR, lo que simplifica considerablemente el proceso. Como Initializr también emplea Maven para generar los archivos, el resultado es el mismo que si lo haces de forma manual.

Para iniciar un proyecto Spring Boot:

- Ingresar al sitio web **Spring Initializr** en start.spring.io.
- Configurar el proyecto, incluyendo las dependencias (en este caso comenzaremos con la dependencia Spring Web).

Para la creación de un proyecto Spring Boot con Spring Initializr se debe considerar los siguientes parámetros de configuración que se deberán establecer desde el generador:

1. **Project:** Permite elegir la herramienta de construcción de la aplicación. En Java las dos herramientas más usadas son **Maven** y Gradle. Recomendamos Maven al ser la más extendida.
2. **Language:** Lenguaje de programación que se va a utilizar en la aplicación. Los tres tipos están soportados por la máquina virtual JVM. **Java** es la opción más extendida y tiene mejor soporte de los editores de programación.
3. **Spring Boot:** Versión del Spring Boot a usar. Siempre que se pueda se optará por la última estable, compuesta únicamente por números (sin palabras entre paréntesis).

4. Project Metadata:

Group: Se refiere al descriptor de Maven **groupId**, utilizado para clasificar el proyecto en los repositorios de binarios. Normalmente se suele usar una referencia similar a la de los packages de las clases. Por ejemplo, **com.example** para disponer todas las aplicaciones web en el mismo directorio.

Artifact: Se refiere al otro descriptor de Maven **artifactId**, y por tanto para indicar el nombre del proyecto y del binario resultante. La combinación de **groupId** y **artifactId** (más la versión) identifican inequívocamente a un binario dentro de cualquier organización.

Name: Asignar un nombre al proyecto.

Description: Determina una descripción al proyecto a crear.

Packaging name: Se refiere al nombre principal del paquete de nuestra aplicación.

Packaging: Indica qué tipo de binario se debe construir. Si la aplicación se ejecutará por sí sola se seleccionará **JAR**, éste contiene todas las dependencias dentro de él y se podrá ejecutar con `java -jar binario.jar`. Si por el contrario, la aplicación se ejecutará en un servidor J2EE existente o en un Tomcat ya desplegado se deberá escoger **WAR**.

Java: Se selecciona la versión de Java a usar. En este caso, se recomienda usar la versión de Java más antigua para garantizar la compatibilidad con otras librerías o proyectos que se quieran incluir, así será más probable encontrar documentación existente que siga siendo válida. Reduce el riesgo de toparse con funcionalidades no muy maduras.

Del lado derecho de la pantalla podremos encontrar el botón de buscador de dependencias.

5. Dependencies: Buscador de dependencias con los starters de Spring boot disponibles. Las dependencias más habituales son:

Spring Web: Orientado al desarrollo de aplicación web o microservicios.

Thymeleaf: Incorpora el motor de plantillas para HTML dinámico (sucesor de JSP - Java Server Page).

Spring Data JPA: Capa estándar de acceso a base de datos SQL denominada Java Persistence Api.

Spring Security: Incorporar controles de acceso en base a usuarios y roles sobre URLs de la aplicación. Asimismo, habilita el control de

ejecución de métodos de servicio en base a roles según los estándares J2EE.

Lombok Aporta: Facilita la programación como la creación de @Getters y @Setters automáticamente para las clases que forman parte del conjunto de mensajes.

Mysql/Postgresql: Incluye el JAR que contiene el driver JDBC necesario para configurar la capa de JPA según la base de datos a usar.

The screenshot displays the Spring Initializr configuration interface. It is divided into several sections:

- Project:** Options for Gradle (Groovy, Kotlin) and Maven (selected).
- Language:** Options for Java (selected), Kotlin, and Groovy.
- Spring Boot:** Version selection including 3.2.0 (SNAPSHOT), 3.1.2 (SNAPSHOT), 3.1.1 (selected), 3.0.9 (SNAPSHOT), 3.0.8, 2.7.14 (SNAPSHOT), and 2.7.13.
- Project Metadata:** Fields for Group (com.example), Artifact (demo), Name (demo), Description (Demo project for Spring Boot), and Package name (com.example.demo).
- Packaging:** Options for Jar (selected) and War.
- Java:** Version selection including 20, 17, 11, and 8 (selected).
- Dependencies:** A section with an "ADD ... CTRL + B" button and a "Spring Web" dependency selected, which includes a "WEB" tag and a description: "Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container."

At the bottom, there are three buttons: "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and "SHARE..."

Después de seleccionar los parámetros necesarios se requiere generar el proyecto, para lo cual se tiene que hacer click en el botón "**Generate Ctrl+**" y automáticamente se descargará el archivo **zip** con el nombre del Artifact que contendrá la carpeta con la estructura de la aplicación lista para importar desde el IDE de tu preferencia.

Después de la descarga del archivo, se debe **descomprimir el archivo ZIP** con la finalidad de importar o abrir el proyecto creado.

Una vez extraído el archivo, ya podemos abrirlo en Eclipse IDE, seleccionando la pestaña **File**, luego **Open Projects from File System** y a continuación eligiendo la carpeta del directorio que contiene el proyecto descomprimido.

Configuración de la tecnología de vista

Spring Boot JSP

Las aplicaciones de **Spring Boot JSP** pueden ser más habituales de lo que en un primer momento nos parece . Sí bien es cierto que usar Spring Boot es usar tecnologías modernas que facilitan que despleguemos arquitecturas orientadas a MicroServicios, hay que recordar que siguen existiendo aplicaciones no tan “modernas” y quizás podamos necesitar que se ejecuten dentro de Spring Boot.

Con el proyecto generado en el punto anterior, ya tenemos la primera dependencia necesaria para un proyecto Spring Boot JSP, a continuación, deberíamos integrar también la siguiente dependencia (puede colocarse directamente en el **pom.xml** en el caso de que el proyecto ya esté generado).

JSP requiere la dependencia del lenguaje **JSTL**, basado en etiquetas (Jakarta Standard Tag Library):

```
<dependency>

    <groupId>org.glassfish.web</groupId>

    <artifactId>jakarta.servlet.jsp.jstl</artifactId>

</dependency>
```

También precisa un motor de compilación porque cada fichero JSP se transforma en un servlet que será compilado. Los servlets son clases Java que procesan peticiones y respuestas HTTP.

```
<dependency>

    <groupId>org.apache.tomcat.embed</groupId>
```

```
<artifactId>tomcat-embed-jasper</artifactId>
```

```
<scope>provided</scope>
```

```
</dependency>
```

En el fichero de configuración **application.properties** definimos el prefijo y sufijo de los ficheros .jsp. El primero es la ruta en la que se ubican dentro de la carpeta **/src/main/webapp** y el segundo la extensión.

```
spring.mvc.view.prefix= /WEB-INF/jsp/
```

```
spring.mvc.view.suffix= .jsp
```

De esta manera ya está completa la infraestructura inicial. Luego deberíamos escribir una clase especial denominada «**controlador**» con métodos que serán invocados cuando llamemos a ciertas direcciones vía HTTP.

Spring Boot Thymeleaf

Thymeleaf es una tecnología que permite definir una plantilla y, junto con un modelo de datos, obtener un nuevo documento. Es lo que se conoce como motor de plantillas.

El motor de plantillas (utilizado específicamente aquí para el desarrollo web) se genera para **separar la interfaz de usuario** (Vistas), **de los datos comerciales** (Modelos), pudiendo generar documentos en un formato específico, en este caso el motor de plantillas para el sitio web generará un documento HTML estándar.

Los motores de plantillas (templates engines) leen un fichero de texto que contiene la presentación ya preparada en HTML, e **inserta en él la información dinámica que le ordena el Controlador**, la parte que une la vista con la información.

Veamos un ejemplo para ver las posibilidades de las plantillas, que no acaban, ni mucho menos, en la web. La sintaxis a utilizar depende del motor de plantillas utilizado, pero todos son muy similares. Los motores de plantillas suelen tener un pequeño lenguaje de script que permite generar código dinámico, como listas o cierto comportamiento condicional, pero esto también depende del lenguaje.

Este lenguaje de script es absolutamente mínimo, lo justo para posibilitar ese comportamiento dinámico:

```
<html>
  <body>
    Hola ${nombre}
  </body>
</html>
```

Está claro que este ejemplo, que es una pequeña variación del famoso "Hola Mundo", es bastante simple. Lo que sucederá al procesar este fichero es que el motor de plantillas lo recorrerá, analizará y sustituirá esa "etiqueta clave" **\${nombre}** por el texto/valor que le hayamos indicado, el nombre del visitante, por ejemplo, de forma que tengamos una presentación personalizada.

Básicamente, el motor de plantillas se encarga de recibir una **variable de tipo String llamada nombre**, que luego se va a enviar desde el controlador a la vista y la hará parte del HTML, haciéndolo dinámico. Por lo que los **diferentes usuarios verán diferentes resultados**.

Thymeleaf permite realizar tareas que se conocen como **natural templating**. Es decir, cómo está basada en añadir atributos y etiquetas, sobre todo HTML, va a permitir que nuestras plantillas se rendericen en local, para que luego sean procesadas dentro del motor de plantillas (por lo cual las tareas de diseño y programación se pueden llevar a cabo conjuntamente).

Permite trabajar con varios tipos de expresiones:

Expresiones variables: Son quizás las más utilizadas y hacen referencia a una variable en particular, como por ejemplo **\${variable}**

Expresiones de selección: Son expresiones que nos permiten reducir la longitud de la expresión si pre fijamos un objeto mediante una expresión variable, como por ejemplo ***{objeto}**

Expresiones de enlace: Nos permiten crear URL que pueden tener parámetros o variables, como por ejemplo **@{link}**

ModelMap

Entonces, gracias a Thymeleaf podemos recibir del controlador una variable y mostrarla en nuestro HTML, pero ¿cómo hacemos para enviar esa variable desde nuestro controlador a nuestro HTML?

Para resolver este problema vamos a utilizar el **objeto ModelMap**, este objeto es parte del paquete **org.springframework.ui.ModelMap**.

El objeto ModelMap tiene el método **addAttribute** que nos permite enviar variables a nuestro HTML. La ventaja del objeto ModelMap, es que también nos permite enviar colecciones a nuestro HTML.

El método **addAttribute(String variable, Objeto nombreObjeto)**, recibe dos argumentos:

- Una variable de tipo String, que va ser el identificador que le vamos a poner al objeto o colección y que también es el identificador con el que va a viajar al HTML
- El objeto de Java que queremos enviar al HTML .

Por ejemplo, supongamos que tenemos la siguiente etiqueta en HTML con Thymeleaf, donde en el **th:text** indicamos que va a recibir una variable llamada **nombre**:

```
<p>Hola<span th:text="${nombre}"></p>
```

En el controlador tendremos el siguiente método:

```
@Controller
public class ControladorHola {
    @GetMapping("/")
    String home(ModelMap model) {
        String nombre = "Fernando"
        model.addAttribute("nombre", nombre)
        return "paginaHTML";
    }
}
```

Como podemos ver en el controlador, recibimos como argumento un ModelMap, esto es para que podamos recibir cualquier modelo que venga de la petición y para que podamos enviar los modelos que necesitemos como respuesta de x peticiones.

Usando model.addAttribute(), pasamos dos parámetros:

1. el identificador con el que el objeto va a viajar a la vista, que es "nombre" (recordemos que tiene que coincidir con la variable de Thymeleaf)
2. el objeto en sí.

Entonces cuando se invoque a este controlador desde el servidor, se enviará el modelo “nombre”, lo recibirá la vista gracias a Thymeleaf y mostrará el nombre Fernando.

ModelAndView

ModelAndView es un objeto que contiene tanto el modelo como la vista. El controlador devuelve el objeto ModelAndView y DispatcherServlet resuelve la vista utilizando View Resolvers y View. La vista es un objeto que contiene el nombre de la vista en forma de cadena y el modelo es un mapa para agregar varios objetos.

Ejemplo:

```
@GetMapping("/")
public ModelAndView helloWorld() {
    String message = "Hello World!";
    return new ModelAndView("welcome", "message", message);
}
```

Configurando Log en Spring

Una vez que tenemos nuestra fantástica aplicación realizada con Spring lo normal es que queramos ver cómo se va comportando. Para ello, la manera más fácil es escribir mensajes dentro de la aplicación, explicando por cuáles funciones entra, como se toman las decisiones y, en general, como se va comportando.

Redirigiendo la salida

Spring Boot, como cualquier aplicación, por defecto mostrará todos los mensajes y/o errores por la salida estándar, pero a menudo deseamos guardar esos mensajes en un fichero. Para ello hay varias opciones. La primera y más obvia es redirigir salida estándar y errores a un fichero. Así, en Linux y/o Windows, podríamos ejecutar nuestra aplicación de esta manera:

```
java -jar APP.JAR > PATH/FICHERO.LOG 2>&1
```

Esta manera funciona, pero tendremos que mirar en el fichero generado para ver como va todo, sin posibilidad de ver inmediatamente en nuestra consola como está funcionando el proceso recién lanzado.

Si lo que deseamos es que deje la salida estándar en un fichero, pero, que además, nos muestre en la consola esa misma salida, podremos lanzar el proceso de esta manera:

```
java -jar -Dlogging.file.name=PATH/FICHERO.LOG APP.JAR
```

Por supuesto también podemos poner en nuestro fichero `application.properties` de configuración esa misma propiedad y podremos lanzar el proceso sin el parámetro '-D'

```
logging.file.name=PATH/FICHERO.LOG APP.JAR
```

Si queremos ejecutar el proceso desde maven pondremos este comando:

```
mvnspring-boot:run-Dspring-boot.run.arguments=-logging.file.name=PATH/FICHERO.LOG
```

Configurando nivel de log

Por supuesto los mensajes que deseamos mostrar los podemos escribir con un simple `system.out.print`, pero desde luego no sería muy elegante ni práctico. Siempre es recomendable usar un logger.

Spring utiliza **Commons Logging**, usando por defecto Logback, si bien incluye también configuraciones por defecto para Java Util Logging y Log4J2.

Por defecto Spring Boot muestra los mensajes de nivel **ERROR**-level, **WARN**-level, y **INFO**-level, pero esto se puede cambiar de diferentes maneras.

La más simple es añadiendo el parámetro **--trace** o **--debug** cuando ejecutemos nuestro programa.

```
java -jar APP.JAR --debug
```

Otra manera será especificar en la propiedad 'logging.level.root'. Esto lo haremos añadiendo la línea correspondiente en el fichero `application.properties`, o de otra manera, a la hora de ejecutarlo, añadiendo el parámetro adecuado:

```
java -jar -Dlogging.level.root=DEBUG APP.JAR
```

Una vez más si usamos maven para ejecutar el proceso, debemos escribir el siguiente comando:

```
mvn spring-boot:run -Dspring-boot.run.arguments  
=-logging.level.root=TRACE
```


Por supuesto, se permite especificar niveles de logging por paquetes, así, si queremos que nuestra aplicación que se encuentra en el paquete 'com.example.demo' muestre los mensajes de nivel debug, pero el resto de aplicaciones solo muestren los mensajes de error, escribiremos esta sentencia:

```
java -jar -Dlogging.level.root=
ERROR -Dlogging.level.com.example.demo=
DEBUG APP.JAR
```

Formateando la salida

Spring por defecto muestra los mensajes de error con este formato:

```
2019-03-05 10:57:51.112 INFO 45469 --- [main]
org.apache.catalina.core.StandardEngine : Starting Servlet Engine: Apache
Tomcat/7.0.52
2019-03-05 10:57:51.253 INFO 45469 --- [ost-startStop-1]
o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded
WebApplicationContext
```

Es decir:

- Día y hora, incluyendo milisegundos.
- Nivel de Log: **ERROR**, **WARN**, **INFO**, **DEBUG**, o **TRACE**.
- Identificador o número del proceso
- Un **---** como separador
- El nombre del Thread entre corchetes.
- El nombre de la clase donde se está escribiendo el log.
- El mensaje de log.

También podemos especificar que use colores para cuando muestre en la consola los logs, con la siguiente variable:

```
spring.output.ansi.enabled=true.
```

Así, los mensajes de error los mostrara en ROJO, los de aviso (WARN) en amarillo y los demás, en verde.

Configuración avanzada

Sí Spring Boot encuentra en el classpath un fichero con alguno de estos nombres, lo usará para configurar los loggers.

- logback-spring.xml
- logback.xml
- logback-spring.groovy
- logback.groovy

Configurando el DataSource

Spring Boot utiliza un algoritmo propio para buscar y configurar un DataSource . Esto nos permite obtener fácilmente una implementación de DataSource completamente configurada de forma predeterminada. Además, Spring Boot configura automáticamente un conjunto de conexiones ultrarrápidas , ya sea HikariCP , Apache Tomcat o Commons DBCP , en ese orden, dependiendo de cuáles están en la ruta de clases.

Si bien la configuración automática de DataSource de Spring Boot funciona muy bien en la mayoría de los casos, a veces necesitaremos un mayor nivel de control.

Lo primero que haremos será editar el fichero de configuración del proyecto (application.properties) para personalizarlo:

application.properties

Configuración para el acceso a la Base de Datos

spring.jpa.hibernate.ddl-auto=update

spring.jpa.properties.hibernate.globally_quoted_identifiers=true

Puerto donde escucha el servidor una vez se inicie

server.port=8080

Datos de conexión con la base de datos MySQL

spring.datasource.url=jdbc:mysql://localhost:3306/myapp

spring.datasource.username=myappUser

spring.datasource.password=myappPassword

Hay que tener en cuenta que la propiedad `spring.jpa.hibernate.ddl-auto` se utiliza para que la base de datos se genere automáticamente en cada arranque de la aplicación. Esto nos interesará cuando estemos en desarrollo pero no cuando

queramos desplegarla en producción. Por lo que tendremos que tener cuidado y controlar el valor de dicha propiedad.

none: Para indicar que no queremos que genere la base de datos.

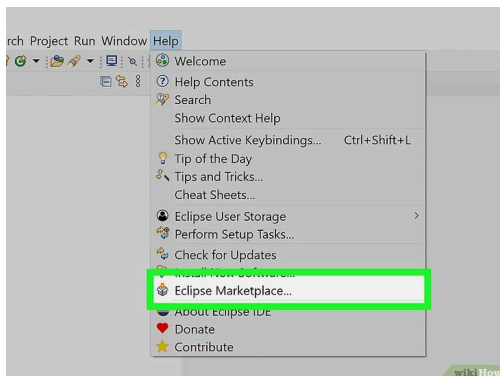
update: Si queremos que la genere de nuevo en cada arranque.

create: Si queremos que cree la DB pero que no la genere de nuevo si ya existe.

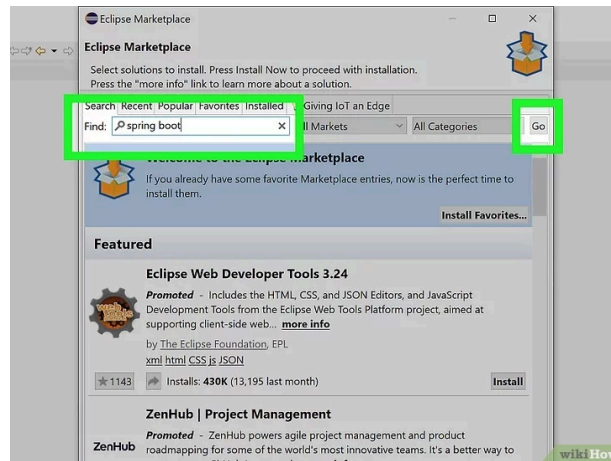
Instalando Spring Tool Suite para Eclipse IDE

En Eclipse, Spring Boot se conoce como Spring Tools Suite. A continuación veremos los pasos para poder instalar las dependencias y componentes necesarios para que el IDE Eclipse pueda correr y compilar correctamente un proyecto Spring.

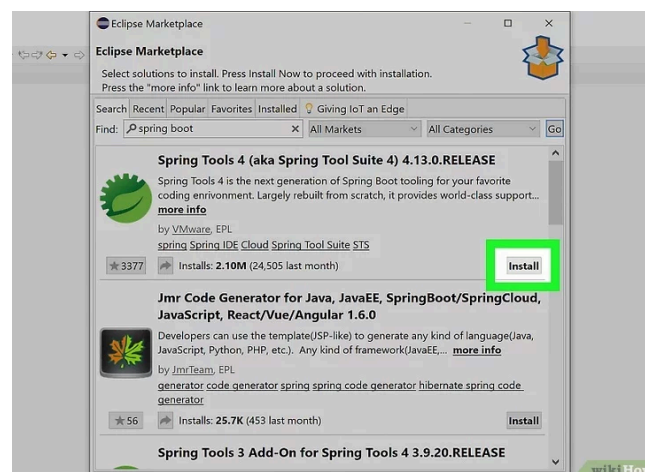
1. Haz clic en **Ayuda**. Es la última opción en la barra de menú ubicada en la parte superior de la pantalla. De esta manera, se mostrará el menú Ayuda.
2. Haz clic en **Eclipse Marketplace**. Se ubica en la parte inferior del menú Ayuda. De esta manera, abrirás el Eclipse Marketplace en una ventana nueva.



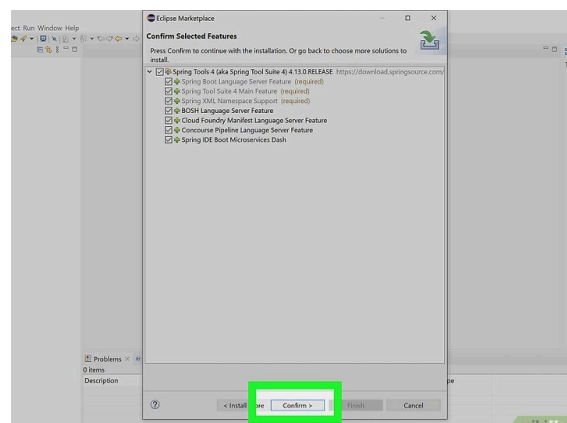
3. Escribe **Spring Boot** en el cuadro de búsqueda y presiona **Enter**. En Eclipse, Spring Boot se conoce como **Spring Tools Suite**.



4. Haz clic en **Instalar** debajo de la versión más reciente de Spring Tools. La versión más reciente de Spring Tools debe aparecer en la parte superior de la lista.



5. Haz clic en **Confirmar**. Se ubica en la parte inferior de la ventana de Eclipse Marketplace. Esto confirmará que quieres instalar todos los paquetes marcados en la lista. Esto iniciará el proceso de instalación de los paquetes seleccionados.



6. Acepta los **Términos y condiciones**, y haz clic en **Finalizar**.
7. Una vez que complete la descarga e instalación, **vuelve a ejecutar Eclipse**. Spring Tools ahora estará instalado y listo para utilizarse.

Capa de Vistas y Controladores

Controllers

Los **Controllers** son las clases Java que usaremos como controladores en nuestra aplicación web. Estas clases son muy parecidas a los **Servlets** de Java EE pero más sencillas de usar. Recuerda que usamos estas clases gracias a que estamos usando el framework Spring MVC (habiendo incluido la dependencia spring-web en nuestro proyecto).

No es necesario que los controladores en Spring MVC extiendan de ninguna otra clase ni que implementen ninguna interfaz. Únicamente deben incluir la anotación **@Controller** al principio de la clase.

```

1. @Controller
2. public class ProfesorController {
3.
4.     @Autowired
5.     private ProfesorDAO profesorDAO;
6.
7.     @RequestMapping("/{index.html}")
8.     public ModelAndView read(HttpServletRequest request, HttpServletResponse response) {
9.         Map<String, Object> model = new HashMap<>();
10.        String viewName;
11.
12.        try {
13.            Profesor profesor = profesorDAO.get(1001);
14.            model.put("texto", profesor.toString());
15.            viewName = "profesor";
16.        } catch (BussinessException ex) {
17.            model.put("msgError", "No es posible obtener los datos");
18.            viewName = "error";
19.        }
20.    }

```

```
21.     return new ModelAndView(viewName, model);  
22. }  
23.  
24.}
```

Pasemos ahora a explicar el código del controlador.

Como ya hemos dicho lo principal que debe tener un controller de Spring MVC es la anotación `@Controller` al principio de la clase (Línea 1). Con esto Spring MVC sabrá que esta clase es un bean de tipo controlador.

Manejo de peticiones

El manejo de peticiones web se realiza a través de los métodos del controlador. Cada método gestionará las peticiones de una o más URLs y puede haber tantos métodos como se desee. Esto es una gran ventaja respecto a un Servlet ya que hace mucho más sencillo procesar una serie de URL relacionadas en un único controlador. Recuerda que en un Servlet había un único método para todas las peticiones.

Cada método que controla peticiones Web tendrá la anotación `@RequestMapping` (Línea 7). Esta anotación tendrá como argumento un Array de Strings con todas las URLs que va a manejar. Por ello nótese que hay una llave “{ }” antes y después del String de la URL que maneja, ya que se permite más de una.

Los argumentos de entrada del método son los objetos `HttpServletRequest` y `HttpServletResponse` al igual que en un Servlet.

Spring MVC tiene muchas funcionalidades relativas a las URL que controla, parámetros de entrada y de salida. Aquí se han utilizado únicamente `HttpServletRequest` y `HttpServletResponse` para asemejarse más a los Servlets y no complicar el uso de Spring MVC, pero se recomienda al programador ver todas las opciones que dispone Spring MVC.

El método retorna una clase de Spring MVC llamada [ModelAndView](#). Esta clase contendrá un String con la página jsp a mostrar (es decir la Vista) y un [Map](#) con los datos que necesita la vista para mostrarse (es decir el Modelo).

Podemos ver en la línea 22: `return new ModelAndView(viewName, model);` como se crea un nuevo objeto `ModelAndView` que tiene como argumentos un `String` con el nombre de la página JSP a mostrar y el `Map` con los datos para la vista.

dispatcher-servlet.xml

Se deberá crear el fichero `dispatcher-servlet.xml` en la carpeta `/WEB-INF`. El contenido del mismo será el siguiente:

1 | dispatcher-servlet.xml

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:context="http://www.springframework.org/schema/context"
5.     xmlns:mvc="http://www.springframework.org/schema/mvc"
6.     xsi:schemaLocation="
7.         http://www.springframework.org/schema/beans
8.         http://www.springframework.org/schema/beans/spring-beans-3.1.xsd
9.         http://www.springframework.org/schema/context
10.        http://www.springframework.org/schema/context/spring-context-3.1.xsd
11.        http://www.springframework.org/schema/mvc
12.        http://www.springframework.org/schema/mvc/spring-mvc-3.1.xsd
13.    ">
14.
15.    <mvc:annotation-driven/>
16.    <context:component-scan base-package="ejemplo02.presentacion" />
17.
18.    <bean
19.        class="org.springframework.web.servlet.view.InternalResourceViewResolver"
20.        >
21.        <property name="prefix" value="/WEB-INF/jsp/" />
22.        <property name="suffix" value=".jsp" />
23.    </bean>
24. </beans>

```

Este código XML hay que copiarlo tal cual está en el `dispatcher-servlet.xml` pero hay 2 líneas que deben modificarse para personalizar la aplicación:

- Línea 13: Se debe indicar el paquete donde se encuentran los controladores web.

- Línea 16: Se indica dónde deben estar las páginas JSP a las que se redirecciona desde los controladores. Se ha elegido `/WEB-INF/jsp/`. Normalmente no es necesario modificar este valor ya que las páginas JSP deberían estar dentro de la carpeta WEB-INF para que desde el navegador no se pueda tener acceso directo a ellas. Debe hacerse a través de los controladores.

La página JSP

Las páginas JSP se deben colocar en la carpeta WEB-INF/jsp tal y como se ha definido en el fichero dispatcher-servlet.xml , en la línea 16.

La vista tiene acceso a los elementos del Map del modelo que se creó en la clase ModelAndView mediante el siguiente código:

```
Object texto=request.getAttribute("texto");
```

Siendo el String `"texto"` la clave del Map con la que se guardó el valor.

Un ejemplo de página JSP es la siguiente:

1 | profesor.jsp

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%
3. Object texto=request.getAttribute("texto");
4. %>
5. <!DOCTYPE html>
6. <html>
7.   <head>
8.     <meta http-equiv="Content-Type" content="text/html;
       charset=UTF-8">
9.     <title>Profesor</title>
10.   </head>
11.   <body>
12.     <h1><%=texto %></h1>
13.   </body>
14. </html>
```

Vemos en la línea 3 cómo se accede a los datos del modelo.

Controladores MultiAction

En el contexto de Spring MVC, un controlador **multiaction** se refiere a una técnica en la cual un único controlador puede manejar múltiples acciones (métodos) basados en diferentes URL o parámetros de solicitud. Esto permite agrupar varias acciones relacionadas en un solo controlador, lo que puede mejorar la organización y la legibilidad del código.

Una de las formas más comunes de implementar un controlador multiaction es utilizar la anotación `@RequestMapping` en los métodos del controlador para mapear diferentes URL o parámetros de solicitud a métodos específicos. Cada método del controlador se encargará de manejar una acción específica.

Aquí hay un ejemplo:

```
@Controller
@RequestMapping("/miControlador")
public class MiControlador {

    @GetMapping("/accion1")
    public String accion1() {
        // Lógica para manejar la acción 1
        return "vista1";
    }

    @PostMapping("/accion2")
    public String accion2() {
        // Lógica para manejar la acción 2
        return "vista2";
    }

    @PostMapping("/accion3")
    public String accion3() {
        // Lógica para manejar la acción 3
        return "vista3";
    }

    // Otros métodos para manejar más acciones
}
```

En este ejemplo, el controlador `MiControlador` tiene tres métodos (`accion1()`, `accion2()`, y `accion3()`) que están mapeados a diferentes URL dentro del contexto `"/miControlador"`. Cuando se recibe una solicitud para `"/miControlador/accion1"`, el método `accion1()` se ejecutará y devolverá la vista `"vista1"`. De manera similar, las URL `"/miControlador/accion2"` y `"/miControlador/accion3"` se mapearán a los métodos `accion2()` y `accion3()`, respectivamente.

Recibiendo datos en el Controlador

En un controlador de Spring, puedes recibir los datos de una vista JSP utilizando la anotación `@RequestParam` o el objeto `HttpServletRequest`. A continuación veremos ejemplos de ambos enfoques:

1. Recepción de datos utilizando la anotación `@RequestParam`:

```
@Controller
public class MiControlador {

    @RequestMapping("/procesarFormulario")
    public String procesarFormulario(@RequestParam("nombre") String
    nombre,@RequestParam("email") String email) {
        // Realizar la lógica de negocio con los datos recibidos
        // Enviar los resultados a la vista o generar una respuesta
        return "vistaJSP";
    }
}
```

En este ejemplo, el método `procesarFormulario()` del controlador `MiControlador` recibe los datos del formulario de la vista JSP utilizando la anotación `@RequestParam`. Cada parámetro anotado representa un campo del formulario con el nombre correspondiente (nombre y email).

`@RequestParam()` es una anotación Spring MVC que se utiliza para vincular los parámetros de una petición HTTP a los argumentos de un método controlador. Por ejemplo, si se envían parámetros de un formulario junto con una URL para guardar un usuario, el método puede obtenerlos como argumentos propios.

2. Recepción de datos utilizando el objeto `HttpServletRequest`:

```
@Controller
public class MiControlador {
```

```

@RequestMapping("/procesarFormulario")
public String procesarFormulario(HttpServletRequest request) {
    String nombre = request.getParameter("nombre");
    String email = request.getParameter("email");
    // Realizar la lógica de negocio con los datos recibidos
    // Pasar los resultados a la vista o generar una respuesta
    return "vistaJSP";
}
}

```

En este ejemplo, el método `procesarFormulario()` del controlador recibe el objeto `HttpServletRequest`, que proporciona acceso a los datos de la solicitud HTTP. Utilizando el método `getParameter()`, puedes obtener los valores de los campos input del formulario HTML por su nombre, como "nombre" y "email".

Enviando datos a la vista

Como hemos visto en el manual, los datos se pueden enviar a la vista de diferentes maneras. En algunos casos se utiliza Model para inyectar los datos en la plantilla, o se puede enviar un objeto ModelAndView directamente a la vista JSP.

Model es una interfaz. Define un contenedor y está diseñado principalmente para agregar atributos al modelo.

1. Usando la interfaz `Model` como parámetro del método del controlador:

```

@Controller
public class MiControlador {
    @RequestMapping("/procesarFormulario")
    public String procesarFormulario(@RequestParam("nombre") String nombre,
    @RequestParam("email") String email, Model model) {
        // Realizar la lógica de negocio con los datos recibidos
        //...
        // Pasar los resultados a la vista a través del objeto Model
        model.addAttribute("resultado", resultado);

        return "vistaJSP";
    }
}

```

En este ejemplo, se utiliza el parámetro Model en el método del controlador. El método `addAttribute()` se utiliza para agregar un atributo, en este caso, el resultado de la lógica de negocio, al objeto Model. Luego, el controlador devuelve el nombre de la vista JSP.

En la vista JSP, puedes acceder al atributo agregado utilizando las expresiones JSP. Por ejemplo:

```
<p>El resultado es: ${resultado}</p>
```

2. Retornando un objeto `ModelAndView` desde el método del controlador:

```
@Controller
public class MiControlador {

    @RequestMapping("/procesarFormulario")
    public ModelAndView procesarFormulario(@RequestParam("nombre") String
nombre, @RequestParam("email") String email) {
        // Realizar la lógica de negocio con los datos recibidos
        // Crear un objeto ModelAndView y agregar los resultados
        ModelAndView modelAndView = new ModelAndView("vistaJSP");
        modelAndView.addObject("resultado", resultado);

        return modelAndView;
    }
}
```

En este ejemplo, se crea un objeto `ModelAndView` en el método del controlador y se establece el nombre de la vista JSP en el constructor. Luego, se utiliza el método `addObject()` para agregar atributos al objeto `ModelAndView`. Finalmente, el controlador devuelve el objeto `ModelAndView`.

En la vista JSP se recibe de la misma manera que en el otro controlador visto:

```
<p>El resultado es: ${resultado}</p>
```

En ambos casos, la vista JSP recibirá el atributo agregado desde el controlador y podrá utilizarlo para mostrar los resultados en la interfaz de usuario.

Desplegando una vista JSP con datos entregados desde el controlador

Para desplegar una vista JSP con los datos entregados desde el controlador en Spring Boot, recomendamos seguir los siguientes pasos:

1. Asegúrate de tener la configuración adecuada para usar JSP en tu aplicación Spring Boot. Puedes hacerlo agregando las siguientes dependencias en tu archivo `pom.xml` si usas Maven:

```

<?xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
</dependency>

```

2. Configura la ubicación de las vistas JSP en tu archivo `application.properties`. Por ejemplo, si tus vistas JSP se encuentran en la carpeta `webapp/views`, agrega la siguiente propiedad en tu archivo `application.properties`:

```

spring.mvc.view.prefix=/views/
spring.mvc.view.suffix=.jsp

```

3. En tu controlador, cuando quieras mostrar la vista JSP con los datos, puedes utilizar el objeto `ModelAndView` como se mencionó anteriormente, o simplemente devolver el nombre de la vista como String, habiendo inyectado el `Model` con los datos necesarios. Por ejemplo:

```

@Controller
public class MiControlador {

    @RequestMapping("/procesarFormulario")
    public String procesarFormulario(@RequestParam("nombre") String nombre,
    @RequestParam("email") String email, ModelAndView model) {

```

```
// Realizar la lógica de negocio con los datos recibidos

// Pasar los resultados a la vista a través del objeto Model
model.addAttribute("resultado", resultado);

return "vistaJSP";
}
}
```

En este ejemplo, el controlador devuelve el nombre de la vista JSP como "vistaJSP".

4. Crea la vista JSP correspondiente en la ubicación configurada anteriormente. Por ejemplo, si configuraste en tu `application.properties` `spring.mvc.view.prefix=/views/`, crea un archivo llamado "vistaJSP.jsp" en la carpeta "webapp/views/".

Al ejecutar la solicitud y el procesamiento en el controlador, Spring Boot buscará la vista JSP correspondiente, reemplazará las expresiones JSP con los valores proporcionados y devolverá el contenido generado al cliente.

Es importante mencionar que el servidor de aplicaciones debe estar correctamente configurado para servir las vistas JSP. Además, la estructura de carpetas y la ubicación de las vistas pueden variar dependiendo de tu configuración específica de la aplicación.

Capa de Servicios

En el patrón de arquitectura Modelo-Vista-Controlador (MVC), la **capa de servicio** (también conocida como **capa de negocio** o **capa de lógica de negocio**) es responsable de implementar la lógica del negocio y coordinar las operaciones entre la capa de presentación (vista) y la capa de acceso a datos (modelo).

El propósito principal de la capa de servicio es encapsular la lógica del negocio y proporcionar una interfaz que la capa de presentación pueda utilizar para acceder y manipular los datos. Esta capa se encarga de procesar las solicitudes recibidas desde la capa de presentación, realizar las operaciones requeridas y devolver los resultados correspondientes.

Algunas de las responsabilidades típicas de la capa de servicio son:

Validación de datos: La capa de servicio valida los datos ingresados por el usuario antes de realizar cualquier operación en la capa de acceso a datos. Esto garantiza que los datos sean consistentes y cumplan con las reglas de negocio establecidas.

Lógica de negocio: La capa de servicio implementa la lógica del negocio de la aplicación. Esto puede incluir cálculos complejos, verificaciones de seguridad, reglas de flujo de trabajo, procesamiento de pagos, integraciones con servicios externos, entre otros.

Coordinación de la capa de acceso a datos: La capa de servicio interactúa con la capa de acceso a datos para realizar operaciones de lectura y escritura en la base de datos u otros sistemas de almacenamiento. Puede orquestar múltiples operaciones de acceso a datos y garantizar la integridad de los datos.

Transacciones y gestión de errores: La capa de servicio puede administrar transacciones, asegurando la integridad de los datos en operaciones que requieren cambios en varias entidades o tablas. También puede manejar excepciones y errores, proporcionando mecanismos para recuperarse de fallos y comunicar mensajes de error adecuados a la capa de presentación.

En resumen, la capa de servicio en el patrón MVC desempeña un papel crucial al implementar la lógica del negocio y actuar como intermediario entre la capa de presentación y la capa de acceso a datos.

Creando un servicio utilizando anotaciones

La creación de servicios utilizando anotaciones puede variar dependiendo del lenguaje de programación o el framework específico que estés utilizando. A continuación, veremos un ejemplo de cómo crear un servicio utilizando anotaciones en el lenguaje de programación Java y el framework Spring.

Supongamos que tienes una clase llamada "**UserService**" que implementa la lógica de negocio para manejar operaciones relacionadas con usuarios. Para convertir esta clase en un servicio utilizando anotaciones deberías hacer lo siguiente:

1. **Anotar la clase:** Agrega la anotación **@Service** en la clase para indicar que es un servicio.

```
@Service
public class UserService {
    // Implementación de la lógica del negocio
    // ...
}
```

- 2. Inyectar dependencias:** Si el servicio necesita utilizar otras clases o componentes, puedes inyectarlos utilizando la anotación `@Autowired`. Por ejemplo:

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    // Implementación de la lógica del negocio utilizando el repositorio
    // ...
}
```

En este ejemplo, se inyecta un objeto `UserRepository` en el servicio. La anotación `@Autowired` se encargará de proporcionar automáticamente una instancia del repositorio cuando se cree una instancia del servicio.

Inyectando el servicio a un controlador para su utilización

Puedes utilizar el servicio en otras partes de tu aplicación inyectándolo de manera similar a como se hizo con las dependencias. Por ejemplo, puedes inyectar el servicio en un controlador utilizando la anotación `@Autowired`:

En este caso, el servicio `UserService` se inyecta en el controlador `UserController` para poder utilizar sus métodos y lógica de negocio.

```
@Controller
public class UserController {

    private UserService userService;

    @Autowired
    public UserController(UserService userService) {
        this.userService = userService;
    }
}
```



```
}  
// Controladores y métodos que utilizan el servicio  
// ...  
}
```

En este ejemplo, se inyecta un objeto `UserService` en el constructor del controlador.

Test y Empaquetamiento en Spring

Test

Spring Framework proporciona soporte integral para realizar **pruebas unitarias e integradas** de aplicaciones. Algunas de las opciones comunes para realizar pruebas en Spring son las siguientes:

- 1. Pruebas unitarias con JUnit:** Puedes utilizar el framework de pruebas unitarias JUnit para escribir pruebas unitarias de componentes individuales, como clases de servicio o controladores. Spring brinda anotaciones como `@RunWith(SpringRunner.class)` y `@SpringBootTest` para facilitar la configuración del contexto de Spring en las pruebas unitarias y la inyección de dependencias.
- 2. Pruebas de integración con Spring Test:** Puedes realizar pruebas de integración para verificar el funcionamiento conjunto de múltiples componentes en un contexto de Spring. Spring Test ofrece otras anotaciones útiles, como `@MockBean` para simular componentes dependientes y `@AutoConfigureMockMvc` para probar controladores web.
- 3. Pruebas de extremo a extremo:** También puedes realizar pruebas de extremo a extremo para verificar el comportamiento de la aplicación en un entorno similar al de producción. En Spring, puedes utilizar herramientas como `RestTemplate` o `WebTestClient` para realizar solicitudes HTTP y validar respuestas en pruebas de extremo a extremo.

Empaquetamiento

El empaquetamiento se refiere a la forma en que se construye y distribuye una aplicación Spring en un archivo ejecutable, como un archivo **JAR** o un archivo **WAR**. Spring ofrece diferentes opciones de empaquetamiento para adaptarse a los diferentes requisitos de implementación:

1. **Archivo JAR ejecutable:** Puedes empaquetar una aplicación Spring en un archivo JAR ejecutable que incluya todas las dependencias necesarias y que pueda ser ejecutado directamente mediante el comando `java -jar`. Para ello, se utiliza el plugin Maven correspondiente para generar el JAR ejecutable.
2. **Archivo WAR:** Si estás desarrollando una aplicación web basada en Spring, puedes empaquetarla en un archivo WAR (Web Application Archive). Este archivo contiene la aplicación web, junto con las dependencias y los recursos necesarios para ser desplegados en un servidor de aplicaciones compatible con `Java EE`.
3. **Empaquetamiento personalizado:** En algunos casos, es posible que necesites un empaquetamiento más personalizado para cumplir con requisitos específicos. Spring ofrece flexibilidad para personalizar el proceso de empaquetamiento según tus necesidades, utilizando herramientas como el plugin Maven Shade o el plugin Gradle Shadow.

Creando unidades de prueba con Spring

Spring proporciona un conjunto de características y anotaciones que facilitan la escritura de pruebas unitarias y de integración para componentes específicos, como servicios y controladores.

Aquí tienes un ejemplo de cómo crear una unidad de prueba para un servicio utilizando Spring:

Supongamos que tienes una clase de servicio llamada `UserService` que tiene métodos para crear y obtener usuarios. Para probar esta clase, puedes seguir estos pasos:

1. **Configuración del contexto de prueba:** Anota tu clase de prueba con `@RunWith(SpringRunner.class)` para indicar que se debe utilizar el runner de Spring para ejecutar las pruebas. También puedes utilizar la anotación `@SpringBootTest` para cargar el contexto de Spring en la prueba. Por ejemplo:

```
@RunWith(SpringRunner.class)
@SpringBootTest
public class UserServiceTest {
    // lógica de la clase test
}
```

2. **Inyección de dependencias:** Utiliza la anotación `@Autowired` para inyectar el servicio `UserService` en la prueba. De esta manera, tendrás acceso al servicio y podrás invocar sus métodos durante la prueba. Por ejemplo:

```
@RunWith(SpringRunner.class)
@SpringBootTest
public class UserServiceTest {

    @Autowired
    private UserService userService;

}
```

3. **Escritura de pruebas:** Escribe métodos de prueba para verificar el comportamiento de los métodos en tu clase de servicio. Puedes utilizar las aserciones proporcionadas por una biblioteca de pruebas, como JUnit o AssertJ, para verificar los resultados esperados. Por ejemplo:

```
@RunWith(SpringRunner.class)
@SpringBootTest
public class UserServiceTest {

    @Autowired
    private UserService userService;

    @Test
    public void testCreateUser() {
        // Lógica de prueba
        // Verificación de resultados utilizando aserciones
    }
}
```

```

@Test
public void testGetUser() {
    // Lógica de prueba
    // Verificación de resultados utilizando aserciones
}
}

```

En estos métodos de prueba, puedes llamar a los métodos correspondientes en el servicio `UserService` y luego verificar si los resultados son los esperados utilizando las **aserciones**.

Este es solo un ejemplo básico de cómo crear una unidad de prueba para un servicio con Spring. Dependiendo de tus necesidades, puedes utilizar otras anotaciones y herramientas proporcionadas por Spring, como `@MockBean` para simular dependencias y `@Autowired` para inyectar esas dependencias simuladas.

Recuerda que las pruebas unitarias deben cubrir diferentes escenarios y casos, y deben verificar el comportamiento esperado de tus componentes.

Empaquetando una aplicación Spring MVC en un archivo WAR

Para empaquetar una aplicación Spring MVC en un archivo WAR (Web Application Archive) que pueda ser desplegado en un servidor de aplicaciones compatible con Java EE recomendamos realizar los siguientes pasos:

1. **Configuración del proyecto:** Esto incluye tener las dependencias adecuadas en tu archivo pom.xml si estás utilizando Maven.
2. **Configuración del archivo de inicio (entry point):** Asegúrate de tener una clase que actúe como punto de entrada (entry point) para tu aplicación Spring MVC. Esta clase generalmente extenderá `AbstractAnnotationConfigDispatcherServletInitializer` y configurará el contexto de la aplicación y el `DispatcherServlet`. Por ejemplo:

```

public class MyWebAppInitializer extends
AbstractAnnotationConfigDispatcherServletInitializer {

    @Override

```

```

protected Class<?>[] getRootConfigClasses() {
    return new Class<?>[]{AppConfig.class};
}
@Override
protected Class<?>[] getServletConfigClasses() {
    return new Class<?>[]{WebConfig.class};
}
@Override
protected String[] getServletMappings() {
    return new String[]{"/*"};
}
}

```

En este ejemplo, `AppConfig` y `WebConfig` son las clases de configuración de Spring que definen los beans y la configuración específica de la aplicación.

3. **Configuración del archivo de descriptores:** Verifica que los archivos de descriptores de implementación de servlets (`web.xml`) y de despliegue (`weblogic.xml`, `jboss-web.xml`, etc, según el servidor de aplicaciones) estén configurados correctamente en tu proyecto. Estos archivos son específicos del servidor de aplicaciones y deben definir el contexto y las configuraciones necesarias para el despliegue.
4. **Empaquetamiento del archivo WAR:** Para empaquetar tu aplicación Spring MVC en un archivo WAR, debes construir el archivo WAR utilizando la herramienta de construcción que estés utilizando, en nuestro caso Maven.

En este caso, se ejecuta el comando `mvn clean package` en el directorio raíz de tu proyecto. Esto compilará el código fuente, ejecutará las pruebas y creará el archivo WAR en el directorio `target` de tu proyecto.

5. **Archivo WAR resultante:** Después de que se complete el proceso de empaquetamiento, encontrarás el archivo **WAR** resultante en el directorio de destino especificado. Este archivo WAR puede ser desplegado en un servidor de aplicaciones compatible con Java EE, como Apache Tomcat, JBoss, WebLogic, entre otros.

Recuerda que los pasos pueden variar ligeramente dependiendo de la configuración y las herramientas específicas que estés utilizando en tu proyecto. Asegúrate de consultar la documentación oficial de Spring y las guías de

construcción de proyectos específicas para obtener más detalles sobre el empaquetamiento de aplicaciones Spring MVC en un archivo WAR.

Cierre

Durante esta lección, exploramos cómo Spring Framework actúa como soporte para el desarrollo de aplicaciones Java, manejando la infraestructura para permitirte enfocarte en tu lógica de negocio.

Además, aprendimos sobre Spring Boot, un framework potente que simplifica la configuración y desarrollo de aplicaciones Java, facilitando su despliegue y distribución.

También abordamos la arquitectura MVC y cómo Spring MVC te ayuda a implementarla, asegurando una clara separación de responsabilidades entre modelo, vista y controlador.

Por último, aprendimos a interactuar con vistas JSP, empaquetar y testear las aplicaciones.

¡Nos vemos de nuevo, más adelante! 

Referencias

- Pahíno, R. (s.f.). ¿Qué son Spring Framework y Spring Boot? Tu primer programa Java con este framework. CampusMVP. Recuperado de <https://www.campusmvp.es/recursos/post/que-son-spring-framework-y-spring-boot-tu-primer-programa-java-con-este-framework.aspx>
- mvit-Innovación Tecnológica. (6 de febrero de 2020). Guía de anotaciones de Spring Framework. Recuperado de <https://mvitinnovaciontecnologica.wordpress.com/2020/02/06/guia-de-anotaciones-de-spring-framework/>
- Arquitectura Java. (s.f.). Spring Stereotypes y anotaciones. Arquitectura Java. Recuperado de <https://www.arquitecturajava.com/spring-stereotypes/#:~:text=Spring%20define%20un%20conjunto%20de...>
- Hadsonpar. (2023, enero 2023). Creando proyectos con Spring Initializr. Hadsonpar Blog. Recuperado de <http://blog.hadsonpar.com/2023/01/creando-proyectos-con-spring-initializr.html>
- Danielme. (2022, 24 de agosto). Tips Spring Boot: integración con JSP. Danielme.com. Recuperado de <https://danielme.com/2022/08/24/tips-spring-boot-integracion-con-jsp/>
- Thymeleaf. (s.f.). Thymeleaf: Java template engine. Recuperado de <https://www.thymeleaf.org/>
- The Apache Software Foundation. (2025, 31 de enero). Apache Commons Logging: User Guide & project overview. Apache Commons. Recuperado de <https://commons.apache.org/proper/commons-logging/>
- Baeldung. (s.f.). Logging in Spring Boot. Baeldung. Recuperado de <https://www.baeldung.com/spring-boot-logging>
- Baeldung. (s.f.). Deploy a Spring Boot WAR into a Tomcat Server. Baeldung. Recuperado de <https://www.baeldung.com/spring-boot-war-tomcat-deploy>
- Álvarez Caules, C. (2018, 20 de noviembre). Spring Boot JSP y su configuración. Arquitectura Java. Recuperado de <https://www.arquitecturajava.com/spring-boot-jsp-y-su-configuracion/>
- Álvarez Caules, C. (2018, 20 de noviembre). Spring Boot JSP y su configuración. Arquitectura Java. Recuperado de <https://www.arquitecturajava.com/spring-boot-jsp-y-su-configuracion/>
- Roales González, N. (2016, 22 de diciembre). Aplicaciones web con Spring Boot capa a capa. Adictos al Trabajo. Recuperado de [https://adictosaltrabajo.com/2016/12/22/aplicaciones-web-con-spring-boo t-capas-a-capas/](https://adictosaltrabajo.com/2016/12/22/aplicaciones-web-con-spring-boot-capa-a-capas/)

¡Muchas gracias!

Nos vemos en la próxima lección

